

UPI PAYMENT SYSTEM PROJECT

=====

Team Members:

+	-----+	+	-----+	+	-----+
	Name		ID Number		
+	-----+	+	-----+	+	-----+
	Deepanshu Garg		2021B3A72758H		
	Arihant Rai		2021B3A72507H		
	Tanmay Agarwal		2021B3A73040H		
	Yash Mohta		2021B3A70569H		
	Kush Desai		2021B4A73158H		
+	-----+	+	-----+	+	-----+

Project Description:

This project implements a secure UPI (Unified Payments Interface) payment system with the following components:

1. Bank Server (bank.cpp):

- Manages user and merchant accounts
- Handles registration and authentication
- Processes transactions
- Maintains blockchain-based transaction ledger
- Supports multiple banks (HDFC, ICICI, SBI) with multiple branches

2. UPI Machine (upi_machine.cpp):

- Acts as intermediary between users/merchants and bank
- Implements SPECK encryption for secure merchant ID handling
- Generates virtual merchant IDs (VMID) for QR codes
- Validates transactions before forwarding to bank

3. User Interface (user.cpp):

- Allows user registration with name, IFSC, password, and initial balance
- Generates secure UUIDs using SHA-256 hashing
- Supports QR code-based payments
- Provides balance checking functionality
- Implements secure transaction PIN system

4. Merchant Interface (merchant.cpp):

- Handles merchant registration and authentication
- Generates payment QR codes
- Receives real-time payment notifications
- Provides balance checking functionality

Security Features:

- SHA-256 hashing for passwords and PINs
- SPECK encryption for merchant IDs
- Blockchain-based transaction logging
- Secure communication between components
- Transaction PIN verification
- Virtual merchant IDs for QR codes

Steps to Run the Project:

1. Ensure you have the following prerequisites:

- C++ compiler with C++17 support
- OpenSSL library
- nlohmann/json library
- QR code generation library

2. Compile the code:

```
...  
g++ -std=c++17 bank.cpp -o bank -lcrypto  
g++ -std=c++17 upi_machine.cpp -o upi_machine  
g++ -std=c++17 merchant.cpp -o merchant -lcrypto  
g++ -std=c++17 user.cpp -o user -lcrypto  
...
```

3. Start the components in the following order:

a. Start the bank server:

```
...  
./bank  
...
```

b. Start the UPI machine:

```
...  
./upi_machine  
...
```

c. Start merchant interface:

```
...  
./merchant  
...
```

d. Start user interface:

```
...  
./user  
...
```

4. Initial Setup:

- a. Register a merchant account
- b. Register a user account
- c. Generate a payment QR code from merchant interface
- d. Make payment from user interface

Note: The system uses the following default ports:

- Bank Server: 9000
- UPI Machine: 9001
- Merchant Notifications: 9100

File Structure:

- bank.cpp: Bank server implementation
- upi_machine.cpp: UPI machine implementation
- merchant.cpp: Merchant interface
- user.cpp: User interface
- merchants.json: Merchant account database
- users.json: User account database
- blockchain.json: Transaction ledger
- transactions.log: Transaction history
- qr_content.txt: QR code data for scanning simulation
- merchant_payment_qr.png: Generated QR code image

Error Handling:

- Invalid balance format
- Insufficient funds
- Invalid credentials
- Connection failures
- Invalid transaction PINs
- Invalid IFSC codes
- QR code generation failures

For any issues or questions, please contact the team members listed above.